Machine Learning Practicals

Complete Answers - All 30 Programs

Program 1: Student Performance Prediction

Problem Statement

A school wants to analyze student performance and predict whether a student will pass or fail based on academic and personal attributes. Develop a machine learning model using the Student Performance Dataset to perform classification. Split the dataset into training and testing sets, build the model, and evaluate its performance using accuracy, precision, recall, and confusion matrix.

Python Code

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score,
confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

# Load dataset
df = pd.read_csv('student-mat.csv', sep=';')

# Create binary target: pass if G3 >= 10
df['pass'] = (df['G3'] >= 10).astype(int)

# Select features
features = ['age', 'absences', 'G1', 'G2', 'studytime', 'failures']
X = df[features]
y = df['pass']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Build model
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Evaluation
print('Accuracy :', accuracy_score(y_test, y_pred))
print('Precision:', precision_score(y_test, y_pred))
print('Recall   :', recall_score(y_test, y_pred))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
```

```
            xticklabels=['Fail','Pass'], yticklabels=['Fail','Pass'])
plt.title('Confusion Matrix - Student Performance')
plt.xlabel('Predicted'); plt.ylabel('Actual')
plt.show()
```

Program 2: Spam Email Detection

Problem Statement

An email service provider needs to automatically detect spam messages. Using the SMS Spam Collection, convert text messages into numerical features and build a classification model. Compare the training error and testing error to analyze model performance.

Python Code

```
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load dataset
df = pd.read_csv('SMSSpamCollection', sep='\t', header=None, names=['label','message'])
df['label_num'] = (df['label'] == 'spam').astype(int)

# Vectorize text
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(df['message'])
y = df['label_num']

# Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train model
model = MultinomialNB()
model.fit(X_train, y_train)

# Errors
train_acc = accuracy_score(y_train, model.predict(X_train))
test_acc  = accuracy_score(y_test,  model.predict(X_test))
print(f'Training Accuracy : {train_acc:.4f}  | Training Error : {1-train_acc:.4f}')
print(f'Testing  Accuracy : {test_acc:.4f}  | Testing  Error : {1-test_acc:.4f}')
```

Program 3: Overfitting vs Underfitting

Problem Statement

A data scientist needs to understand the impact of model complexity on performance. Using the Wine Quality Dataset, train both simple and complex models. Compare their performance and identify cases of overfitting and underfitting.

Python Code

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Load dataset
df = pd.read_csv('winequality-red.csv', sep=';')
X = df.drop('quality', axis=1)
y = df['quality']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print(f'{'Depth':<10} {'Train Acc':<15} {'Test Acc':<15} {'Observation'}')
print('-' * 60)

for depth in [1, 3, 5, 10, None]:
    model = DecisionTreeClassifier(max_depth=depth, random_state=42)
    model.fit(X_train, y_train)
    tr = accuracy_score(y_train, model.predict(X_train))
    te = accuracy_score(y_test,  model.predict(X_test))
    obs = 'Underfitting' if depth == 1 else ('Overfitting' if depth is None else 'Good
fit')
    print(f'{str(depth):<10} {tr:<15.4f} {te:<15.4f} {obs}')
```

Program 4: Gradient Descent Implementation

Problem Statement

A real estate company wants to predict house prices using regression. Using the Boston Housing Dataset, implement linear regression using gradient descent and visualize the reduction in error over iterations.

Python Code

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing  # Boston deprecated; using
California
from sklearn.preprocessing import StandardScaler

# Load data
data = fetch_california_housing()
X, y = data.data[:, 0].reshape(-1,1), data.target  # Use 1 feature for simplicity

# Normalize
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Gradient Descent
m, b = 0.0, 0.0
lr = 0.01
n = len(y)
errors = []

for i in range(1000):
    y_pred = m * X_scaled.flatten() + b
```

```
    error  = y_pred - y
    mse    = np.mean(error**2)
    errors.append(mse)
    m -= lr * (2/n) * np.dot(error, X_scaled.flatten())
    b -= lr * (2/n) * np.sum(error)

print(f'Final m={m:.4f}, b={b:.4f}, MSE={errors[-1]:.4f}')

# Plot error over iterations
plt.plot(errors)
plt.title('Gradient Descent - MSE over Iterations')
plt.xlabel('Iteration'); plt.ylabel('MSE')
plt.show()
```

Program 5: Bias-Variance Analysis

Problem Statement

To study model generalization, generate a synthetic dataset using sklearn and train models with varying complexity. Analyze and explain the bias-variance trade-off based on model performance.

Python Code

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_regression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

X, y = make_regression(n_samples=200, n_features=1, noise=20, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print(f'{'Degree':<10} {'Train MSE':<15} {'Test MSE':<15} {'Observation'}')
print('-' * 60)

for deg in [1, 2, 5, 10, 15]:
    pipe = make_pipeline(PolynomialFeatures(deg), LinearRegression())
    pipe.fit(X_train, y_train)
    tr_mse = mean_squared_error(y_train, pipe.predict(X_train))
    te_mse = mean_squared_error(y_test,  pipe.predict(X_test))
    if deg == 1:
        obs = 'High Bias (Underfitting)'
    elif deg <= 3:
        obs = 'Good Balance'
    else:
        obs = 'High Variance (Overfitting)'
    print(f'{deg:<10} {tr_mse:<15.2f} {te_mse:<15.2f} {obs}')
```

Program 6: Disease Prediction Evaluation

Problem Statement

A healthcare organization wants to predict the presence of disease in patients. Using the Breast Cancer Wisconsin Dataset, build a classification model and evaluate it using confusion matrix, precision, and recall. Analyze which metric is more important in this scenario.

Python Code

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, precision_score, recall_score,
classification_report
import seaborn as sns
import matplotlib.pyplot as plt

data = load_breast_cancer()
X, y = data.data, data.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = LogisticRegression(max_iter=10000)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print(classification_report(y_test, y_pred, target_names=['Malignant','Benign']))

cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Reds',
            xticklabels=['Malignant','Benign'],
            yticklabels=['Malignant','Benign'])
plt.title('Confusion Matrix - Breast Cancer Detection')
plt.show()

# Analysis
print('In disease detection, RECALL is more important.')
print('A False Negative (missed cancer) is far more dangerous than a False Positive.')
```

Program 7: Handling Missing Values

Problem Statement

A dataset used for survival prediction contains missing values. Using the Titanic Dataset, identify missing values and handle them using mean, median, or mode. Compare the dataset before and after preprocessing.

Python Code

```
import pandas as pd
import seaborn as sns

# Load Titanic dataset
df = sns.load_dataset('titanic')

print('=== BEFORE Preprocessing ===')
```

```
print(df.isnull().sum())

# Handle missing values
df['age'].fillna(df['age'].median(), inplace=True)          # median for age
df['embarked'].fillna(df['embarked'].mode()[0], inplace=True)  # mode for embarked
df.drop(columns=['deck'], inplace=True)                     # too many nulls

print('\n=== AFTER Preprocessing ===')
print(df.isnull().sum())
print('\nDataset shape:', df.shape)
```

Program 8: Noisy Data Cleaning

Problem Statement

Environmental sensor data often contains noise and irregularities. Using the Air Quality Dataset, detect noisy values and apply appropriate techniques such as smoothing or filtering to clean the data.

Python Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load Air Quality dataset (download from UCI)
df = pd.read_csv('AirQualityUCI.csv', sep=';', decimal=',')
df = df.iloc[:, :4]  # Use first 4 columns
df.columns = ['Date','Time','CO_GT','PT08']
df['CO_GT'] = pd.to_numeric(df['CO_GT'], errors='coerce')
df.dropna(inplace=True)
df = df[df['CO_GT'] != -200]  # Remove sentinel missing values

# Detect outliers using IQR
Q1, Q3 = df['CO_GT'].quantile(0.25), df['CO_GT'].quantile(0.75)
IQR = Q3 - Q1
lower, upper = Q1 - 1.5*IQR, Q3 + 1.5*IQR
print(f'Outliers detected: {((df["CO_GT"] < lower) | (df["CO_GT"] > upper)).sum()}')

# Clip outliers
df['CO_GT_clean'] = df['CO_GT'].clip(lower, upper)

# Smoothing with rolling mean
df['CO_GT_smooth'] = df['CO_GT_clean'].rolling(window=5, min_periods=1).mean()

# Plot
plt.figure(figsize=(12,4))
plt.plot(df['CO_GT'].values[:200], label='Original', alpha=0.5)
plt.plot(df['CO_GT_smooth'].values[:200], label='Smoothed', linewidth=2)
plt.title('Air Quality CO - Before and After Smoothing')
plt.legend(); plt.show()
```

Program 9: Data Normalization

Problem Statement

A dataset contains features with varying scales. Using the Adult Income Dataset, apply Min-Max normalization and Z-score normalization. Compare the transformed outputs.

Python Code

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler, StandardScaler

cols = ['age','fnlwgt','education-num','hours-per-week','capital-gain','capital-loss']
url  = 'https://archive.ics.uci.edu/ml/machine-learning-databases/adult/adult.data'
df   = pd.read_csv(url, header=None,
                   names=['age','workclass','fnlwgt','education','education-num',
                          'marital-status','occupation','relationship','race',
                          'sex','capital-gain','capital-loss','hours-per-week',
                          'native-country','income'])

X = df[cols]

# Min-Max Normalization
mm_scaler = MinMaxScaler()
X_minmax  = pd.DataFrame(mm_scaler.fit_transform(X), columns=cols)

# Z-score Normalization
z_scaler  = StandardScaler()
X_zscore  = pd.DataFrame(z_scaler.fit_transform(X), columns=cols)

print('=== Original Data Stats ===')
print(X.describe().round(2))
print('\n=== Min-Max Normalized Stats ===')
print(X_minmax.describe().round(4))
print('\n=== Z-score Normalized Stats ===')
print(X_zscore.describe().round(4))
```

Program 10: Encoding Categorical Data

Problem Statement

Machine learning models require numerical input. Using the Car Evaluation Dataset, convert categorical features into numerical format using Label Encoding and One-Hot Encoding. Compare their impact on model performance.

Python Code

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

cols = ['buying','maint','doors','persons','lug_boot','safety','class']
df   = pd.read_csv('car.data', names=cols)

# --- Label Encoding ---
```

```
df_le = df.copy()
le = LabelEncoder()
for c in df_le.columns:
    df_le[c] = le.fit_transform(df_le[c])

X_le = df_le.drop('class', axis=1)
y_le = df_le['class']
Xtr, Xte, ytr, yte = train_test_split(X_le, y_le, test_size=0.2, random_state=42)
acc_le = accuracy_score(yte, DecisionTreeClassifier().fit(Xtr, ytr).predict(Xte))

# --- One-Hot Encoding ---
df_oh = pd.get_dummies(df.drop('class', axis=1))
y_oh  = le.fit_transform(df['class'])
Xtr, Xte, ytr, yte = train_test_split(df_oh, y_oh, test_size=0.2, random_state=42)
acc_oh = accuracy_score(yte, DecisionTreeClassifier().fit(Xtr, ytr).predict(Xte))

print(f'Label Encoding Accuracy  : {acc_le:.4f}')
print(f'One-Hot Encoding Accuracy: {acc_oh:.4f}')
```

Program 11: PCA Dimensionality Reduction

Problem Statement

High-dimensional data can reduce model efficiency. Using the Iris Dataset, apply Principal Component Analysis (PCA) to reduce features to two dimensions and visualize the results.

Python Code

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

data = load_iris()
X, y = data.data, data.target

# Standardize
X_scaled = StandardScaler().fit_transform(X)

# Apply PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

print('Original shape :', X.shape)
print('Reduced shape  :', X_pca.shape)
print('Explained Variance Ratio:', pca.explained_variance_ratio_)
print(f'Total Variance Retained : {sum(pca.explained_variance_ratio_)*100:.2f}%')

# Visualize
colors = ['red','green','blue']
for i, name in enumerate(data.target_names):
    plt.scatter(X_pca[y==i,0], X_pca[y==i,1], label=name, c=colors[i])
plt.xlabel('PC1'); plt.ylabel('PC2')
plt.title('PCA on Iris Dataset')
plt.legend(); plt.show()
```

Program 12: Full Preprocessing Pipeline

Problem Statement

A business dataset requires complete preprocessing before analysis. Using the Superstore Sales Dataset, perform missing value handling, normalization, and categorical encoding to prepare the dataset for modeling.

Python Code

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler, LabelEncoder

# Load dataset
df = pd.read_csv('Sample - Superstore.csv', encoding='latin1')

print('=== Step 1: Missing Values ===')
print(df.isnull().sum()[df.isnull().sum() > 0])
df.fillna(df.select_dtypes(include='number').median(), inplace=True)
print('Missing values handled.')

# Step 2: Categorical Encoding
print('\n=== Step 2: Label Encoding Categorical Columns ===')
le = LabelEncoder()
cat_cols = ['Ship Mode','Segment','Region','Category','Sub-Category']
for col in cat_cols:
    if col in df.columns:
        df[col + '_enc'] = le.fit_transform(df[col].astype(str))
print('Encoded:', cat_cols)

# Step 3: Normalize numerical columns
print('\n=== Step 3: Min-Max Normalization ===')
num_cols = ['Sales','Profit','Quantity','Discount']
existing = [c for c in num_cols if c in df.columns]
scaler = MinMaxScaler()
df[existing] = scaler.fit_transform(df[existing])
print('Normalized:', existing)
print('\nFinal dataset shape:', df.shape)
print(df[existing].describe().round(4))
```

Program 13: Heart Disease Prediction (KNN)

Problem Statement

A hospital wants to predict heart disease in patients. Using the Heart Disease Dataset, implement the K-Nearest Neighbors (KNN) algorithm, experiment with different values of K, and evaluate accuracy.

Python Code

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Load dataset (Cleveland heart disease dataset)
url = 'https://archive.ics.uci.edu/ml/machine-learning-databases/heart-
disease/processed.cleveland.data'
cols = ['age','sex','cp','trestbps','chol','fbs','restecg',
        'thalach','exang','oldpeak','slope','ca','thal','target']
df = pd.read_csv(url, names=cols, na_values='?')
df.dropna(inplace=True)
df['target'] = (df['target'] > 0).astype(int)

X = df.drop('target', axis=1)
y = df['target']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test  = scaler.transform(X_test)

accs = []
k_range = range(1, 21)
for k in k_range:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)
    accs.append(accuracy_score(y_test, knn.predict(X_test)))

best_k = k_range[accs.index(max(accs))]
print(f'Best K: {best_k} with Accuracy: {max(accs):.4f}')

plt.plot(k_range, accs, marker='o')
plt.title('KNN Accuracy vs K - Heart Disease')
plt.xlabel('K'); plt.ylabel('Accuracy')
plt.xticks(k_range); plt.show()
```

Program 14: Diabetes Prediction (Logistic Regression)

Problem Statement

A healthcare system needs to predict diabetes in patients. Using the Pima Indians Diabetes Dataset, apply Logistic Regression and evaluate the model performance.

Python Code

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

url = 'https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-
diabetes.data.csv'
cols = ['Pregnancies','Glucose','BloodPressure','SkinThickness',
        'Insulin','BMI','DiabetesPedigree','Age','Outcome']
df = pd.read_csv(url, names=cols)
```

```
X = df.drop('Outcome', axis=1)
y = df['Outcome']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test  = scaler.transform(X_test)

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print('Accuracy:', accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))

cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Greens')
plt.title('Confusion Matrix - Diabetes Prediction')
plt.show()
```

Program 15: Iris Classification (Decision Tree)

Problem Statement

A botanist wants to classify flowers into species. Using the Iris Dataset, build a Decision Tree classifier and visualize the tree structure.

Python Code

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

data = load_iris()
X, y = data.data, data.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = DecisionTreeClassifier(max_depth=3, random_state=42)
model.fit(X_train, y_train)

print('Accuracy:', accuracy_score(y_test, model.predict(X_test)))

plt.figure(figsize=(12, 6))
plot_tree(model, feature_names=data.feature_names,
          class_names=data.target_names, filled=True, rounded=True)
plt.title('Decision Tree - Iris Classification')
plt.show()
```

Program 16: Spam Detection (Naive Bayes)

Problem Statement

Improve spam detection using probabilistic models. Using the SMS Spam Collection, implement Naive Bayes classification and compare results with previous models.

Python Code

```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

df = pd.read_csv('SMSSpamCollection', sep='\t', header=None, names=['label','message'])
df['label_num'] = (df['label'] == 'spam').astype(int)

# TF-IDF gives better features than simple CountVectorizer
tfidf = TfidfVectorizer(stop_words='english', max_features=3000)
X = tfidf.fit_transform(df['message'])
y = df['label_num']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = MultinomialNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print('=== Naive Bayes with TF-IDF ===')
print(f'Accuracy: {accuracy_score(y_test, y_pred):.4f}')
print(classification_report(y_test, y_pred, target_names=['Ham','Spam']))
```

Program 17: Customer Classification (KNN)

Problem Statement

A retail store wants to classify customers based on behavior. Using the Mall Customer Segmentation Dataset, apply KNN classification and evaluate the results.

Python Code

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report

df = pd.read_csv('Mall_Customers.csv')

# Encode Gender
df['Gender'] = LabelEncoder().fit_transform(df['Genre'])

# Create spending category as target
df['SpendingCat'] = pd.cut(df['Spending Score (1-100)'],
                           bins=[0,33,66,100],
                           labels=['Low','Medium','High'])
```

```
X = df[['Age','Annual Income (k$)','Gender']]
y = df['SpendingCat']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test  = scaler.transform(X_test)

model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print(f'Accuracy: {accuracy_score(y_test, y_pred):.4f}')
print(classification_report(y_test, y_pred))
```

Program 18: Loan Approval Prediction (SVM)

Problem Statement

A bank wants to automate loan approval decisions. Using the Loan Prediction Dataset, implement Support Vector Machine (SVM) and evaluate accuracy.

Python Code

```
import pandas as pd
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import accuracy_score, classification_report

df = pd.read_csv('loan_prediction.csv')

# Preprocessing
df.dropna(inplace=True)
le = LabelEncoder()
cat_cols = ['Gender','Married','Education','Self_Employed','Property_Area','Loan_Status']
for col in cat_cols:
    df[col] = le.fit_transform(df[col])

df['Dependents'] = df['Dependents'].replace('3+', 3).astype(int)

features = ['Gender','Married','Dependents','Education','Self_Employed',
            'ApplicantIncome','CoapplicantIncome','LoanAmount',
            'Loan_Amount_Term','Credit_History','Property_Area']
X = df[features]
y = df['Loan_Status']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test  = scaler.transform(X_test)

model = SVC(kernel='rbf', C=1.0, gamma='scale')
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

```
print(f'SVM Accuracy: {accuracy_score(y_test, y_pred):.4f}')
print(classification_report(y_test, y_pred))
```

Program 19: Credit Risk Prediction (Random Forest vs Decision Tree)

Problem Statement

A financial institution needs to assess credit risk. Using the Credit Card Default Dataset, apply Random Forest and compare its performance with a Decision Tree model.

Python Code

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

# Download from: https://archive.ics.uci.edu/ml/datasets/default+of+credit+card+clients
df = pd.read_excel('default of credit card clients.xls', header=1)
df.rename(columns={'default payment next month': 'DEFAULT'}, inplace=True)

X = df.drop(['ID','DEFAULT'], axis=1)
y = df['DEFAULT']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Decision Tree
dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)
dt_acc = accuracy_score(y_test, dt.predict(X_test))

# Random Forest
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
rf_acc = accuracy_score(y_test, rf.predict(X_test))

print(f'Decision Tree Accuracy : {dt_acc:.4f}')
print(f'Random Forest Accuracy : {rf_acc:.4f}')
print('\nRandom Forest Report:')
print(classification_report(y_test, rf.predict(X_test)))
```

Program 20: House Price Prediction (Linear Regression)

Problem Statement

A real estate company wants to predict housing prices. Using the House Prices Dataset, implement Linear Regression and evaluate error metrics such as MSE or RMSE.

Python Code

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import StandardScaler

# Using California Housing as a substitute for Boston (deprecated)
from sklearn.datasets import fetch_california_housing
data = fetch_california_housing()
df = pd.DataFrame(data.data, columns=data.feature_names)
df['Price'] = data.target

X = df.drop('Price', axis=1)
y = df['Price']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test  = scaler.transform(X_test)

model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

mse  = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2   = r2_score(y_test, y_pred)

print(f'MSE  : {mse:.4f}')
print(f'RMSE : {rmse:.4f}')
print(f'R^2  : {r2:.4f}')
```

Program 21: Customer Segmentation (K-Means)

Problem Statement

A shopping mall wants to segment customers based on purchasing behavior. Using the Mall Customer Segmentation Dataset, apply K-Means clustering and determine the optimal number of clusters.

Python Code

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

df = pd.read_csv('Mall_Customers.csv')
X = df[['Annual Income (k$)', 'Spending Score (1-100)']]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Elbow Method
```

```
inertia = []
K = range(1, 11)
for k in K:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X_scaled)
    inertia.append(km.inertia_)

plt.figure(figsize=(8,4))
plt.plot(K, inertia, 'bo-')
plt.title('Elbow Method - Optimal K')
plt.xlabel('Number of Clusters'); plt.ylabel('Inertia')
plt.show()

# Optimal K = 5 (from elbow)
km = KMeans(n_clusters=5, random_state=42, n_init=10)
df['Cluster'] = km.fit_predict(X_scaled)

colors = ['red','blue','green','purple','orange']
for i in range(5):
    subset = df[df['Cluster'] == i]
    plt.scatter(subset['Annual Income (k$)'], subset['Spending Score (1-100)'],
                color=colors[i], label=f'Cluster {i}')
plt.title('K-Means Customer Segmentation')
plt.xlabel('Annual Income (k$)'); plt.ylabel('Spending Score')
plt.legend(); plt.show()
```

Program 22: Wholesale Customer Clustering

Problem Statement

A wholesale distributor wants to group customers based on purchasing patterns. Using the Wholesale Customers Dataset, perform clustering and analyze the formed groups.

Python Code

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

url = 'https://archive.ics.uci.edu/ml/machine-learning-
databases/00292/Wholesale%20customers%20data.csv'
df  = pd.read_csv(url)

# Use spending features
X = df[['Fresh','Milk','Grocery','Frozen','Detergents_Paper','Delicassen']]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Elbow method
inertia = [KMeans(n_clusters=k, random_state=42, n_init=10).fit(X_scaled).inertia_
           for k in range(1, 11)]
plt.plot(range(1,11), inertia, 'bo-')
plt.title('Elbow - Wholesale Customers'); plt.show()

# Fit with 3 clusters
km = KMeans(n_clusters=3, random_state=42, n_init=10)
```

```
df['Cluster'] = km.fit_predict(X_scaled)

print('=== Cluster Profiles ===')
print(df.groupby('Cluster')[['Fresh','Milk','Grocery','Frozen']].mean().round(1))
```

Program 23: Document Clustering

Problem Statement

A news aggregator wants to group similar articles. Using the 20 Newsgroups Dataset, convert text into numerical vectors and apply clustering techniques.

Python Code

```
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import KMeans
from sklearn.decomposition import TruncatedSVD
import matplotlib.pyplot as plt

# Load subset of categories
categories = ['rec.sport.soccer','sci.space','talk.politics.guns','comp.graphics']
data = fetch_20newsgroups(subset='train', categories=categories,
                          remove=('headers','footers','quotes'))

# TF-IDF Vectorization
tfidf = TfidfVectorizer(max_features=5000, stop_words='english')
X = tfidf.fit_transform(data.data)

# Reduce dims for visualization
svd = TruncatedSVD(n_components=2, random_state=42)
X_2d = svd.fit_transform(X)

# K-Means clustering
km = KMeans(n_clusters=4, random_state=42, n_init=10)
labels = km.fit_predict(X)

# Visualize
plt.figure(figsize=(8,6))
for c in range(4):
    plt.scatter(X_2d[labels==c, 0], X_2d[labels==c, 1],
                label=f'Cluster {c}', alpha=0.5, s=10)
plt.title('Document Clustering - 20 Newsgroups')
plt.legend(); plt.show()

# Top terms per cluster
terms = tfidf.get_feature_names_out()
for i, center in enumerate(km.cluster_centers_):
    top = [terms[j] for j in center.argsort()[-10:][::-1]]
    print(f'Cluster {i}: {top}')
```

Program 24: Hierarchical Clustering

Problem Statement

To understand relationships among data points, use the Iris Dataset to perform hierarchical clustering and plot a dendrogram.

Python Code

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import adjusted_rand_score

data = load_iris()
X = StandardScaler().fit_transform(data.data)

# Dendrogram (using a sample for clarity)
Z = linkage(X[:50], method='ward')
plt.figure(figsize=(12, 5))
dendrogram(Z, leaf_rotation=90, leaf_font_size=8)
plt.title('Hierarchical Clustering Dendrogram (Iris - first 50 samples)')
plt.xlabel('Sample Index'); plt.ylabel('Distance')
plt.show()

# Agglomerative Clustering
model = AgglomerativeClustering(n_clusters=3, linkage='ward')
labels = model.fit_predict(X)

ari = adjusted_rand_score(data.target, labels)
print(f'Adjusted Rand Index: {ari:.4f}')
print('Cluster distribution:', {i: list(labels).count(i) for i in range(3)})
```

Program 25: Cluster and Classify

Problem Statement

A company wants to first group customers and then predict their categories. Perform clustering on a dataset, then use the cluster labels as input for a supervised classification model.

Python Code

```
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report

# Use Mall Customers dataset
df = pd.read_csv('Mall_Customers.csv')
df['Gender_enc'] = (df['Genre'] == 'Male').astype(int)

features = ['Age','Annual Income (k$)','Spending Score (1-100)','Gender_enc']
X = df[features]
```

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Step 1: Clustering
km = KMeans(n_clusters=4, random_state=42, n_init=10)
df['Cluster'] = km.fit_predict(X_scaled)
print('Cluster distribution:', df['Cluster'].value_counts().to_dict())

# Step 2: Classification using cluster labels as target
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, df['Cluster'], test_size=0.2, random_state=42)

clf = RandomForestClassifier(n_estimators=100, random_state=42)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)

print(f'Classification Accuracy: {accuracy_score(y_test, y_pred):.4f}')
print(classification_report(y_test, y_pred))
```

Program 26: Digit Recognition (ANN)

Problem Statement

Develop a system to recognize handwritten digits. Using the MNIST Dataset, build an Artificial Neural Network (ANN) and evaluate its accuracy.

Python Code

```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import mnist
import matplotlib.pyplot as plt

# Load and preprocess data
(X_train, y_train), (X_test, y_test) = mnist.load_data()
X_train = X_train.reshape(-1, 784) / 255.0
X_test  = X_test.reshape(-1, 784)  / 255.0

# Build ANN
model = models.Sequential([
    layers.Dense(128, activation='relu', input_shape=(784,)),
    layers.Dropout(0.2),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

history = model.fit(X_train, y_train, epochs=10,
                    batch_size=128, validation_split=0.1, verbose=1)

test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0)
print(f'Test Accuracy: {test_acc:.4f}')
```

```
# Plot accuracy
plt.plot(history.history['accuracy'], label='Train')
plt.plot(history.history['val_accuracy'], label='Val')
plt.title('ANN - MNIST Accuracy'); plt.legend(); plt.show()
```

Program 27: Student Performance using ANN (MLP)

Problem Statement

Predict student outcomes using neural networks. Using the Student Performance Dataset, implement a Multilayer Perceptron (MLP) model.

Python Code

```
import pandas as pd
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report

df = pd.read_csv('student-mat.csv', sep=';')
df['pass'] = (df['G3'] >= 10).astype(int)

features = ['age','absences','G1','G2','studytime','failures','Medu','Fedu']
X = df[features]
y = df['pass']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test  = scaler.transform(X_test)

# MLP with 2 hidden layers
mlp = MLPClassifier(
    hidden_layer_sizes=(64, 32),
    activation='relu',
    max_iter=500,
    random_state=42
)
mlp.fit(X_train, y_train)
y_pred = mlp.predict(X_test)

print(f'MLP Accuracy: {accuracy_score(y_test, y_pred):.4f}')
print(classification_report(y_test, y_pred))
```

Program 28: Activation Function Comparison

Problem Statement

Different activation functions impact model performance. Using the Fashion MNIST Dataset, train models with Sigmoid, ReLU, and Tanh functions and compare their accuracy.

Python Code

```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import fashion_mnist

(X_train, y_train), (X_test, y_test) = fashion_mnist.load_data()
X_train = X_train.reshape(-1, 784) / 255.0
X_test  = X_test.reshape(-1, 784)  / 255.0

def build_model(activation):
    m = models.Sequential([
        layers.Dense(128, activation=activation, input_shape=(784,)),
        layers.Dense(64,  activation=activation),
        layers.Dense(10,  activation='softmax')
    ])
    m.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
    return m

results = {}
for act in ['sigmoid', 'relu', 'tanh']:
    print(f'\nTraining with {act}...')
    m = build_model(act)
    m.fit(X_train, y_train, epochs=5, batch_size=128,
          validation_split=0.1, verbose=0)
    _, acc = m.evaluate(X_test, y_test, verbose=0)
    results[act] = acc
    print(f'{act} Test Accuracy: {acc:.4f}')

print('\n=== Comparison ===')
for act, acc in results.items():
    print(f'{act:<10}: {acc:.4f}')
```

Program 29: Hyperparameter Tuning

Problem Statement

Optimize neural network performance. Using the CIFAR-10 Dataset, tune hyperparameters such as learning rate and epochs and compare results.

Python Code

```
import tensorflow as tf
from tensorflow.keras import layers, models, optimizers
from tensorflow.keras.datasets import cifar10

(X_train, y_train), (X_test, y_test) = cifar10.load_data()
X_train, X_test = X_train / 255.0, X_test / 255.0

def build_cnn(lr=0.001):
    m = models.Sequential([
        layers.Conv2D(32, (3,3), activation='relu', input_shape=(32,32,3)),
        layers.MaxPooling2D((2,2)),
```

```
        layers.Conv2D(64, (3,3), activation='relu'),
        layers.MaxPooling2D((2,2)),
        layers.Flatten(),
        layers.Dense(64, activation='relu'),
        layers.Dense(10, activation='softmax')
    ])
    m.compile(optimizer=optimizers.Adam(learning_rate=lr),
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
    return m

# Hyperparameter combinations
configs = [
    {'lr': 0.001, 'epochs': 5},
    {'lr': 0.01,  'epochs': 5},
    {'lr': 0.001, 'epochs': 10},
]

for cfg in configs:
    print(f"\nLR={cfg['lr']}, Epochs={cfg['epochs']}")
    model = build_cnn(lr=cfg['lr'])
    model.fit(X_train, y_train, epochs=cfg['epochs'],
              batch_size=64, validation_split=0.1, verbose=0)
    _, acc = model.evaluate(X_test, y_test, verbose=0)
    print(f"Test Accuracy: {acc:.4f}")
```

Program 30: Ensemble Learning (Bagging and Boosting)

Problem Statement

A bank wants to improve prediction accuracy using multiple models. Using the Bank Marketing Dataset, implement Bagging and Boosting techniques and compare their performance.

Python Code

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import BaggingClassifier, GradientBoostingClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report

# Load dataset
url = 'https://archive.ics.uci.edu/ml/machine-learning-databases/00222/bank.zip'
# Or local: df = pd.read_csv('bank.csv', sep=';')
df = pd.read_csv('bank.csv', sep=';')

# Encode categoricals
le = LabelEncoder()
for col in df.select_dtypes(include='object').columns:
    df[col] = le.fit_transform(df[col])

X = df.drop('y', axis=1)
y = df['y']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Bagging
```

```
bagging = BaggingClassifier(
    estimator=DecisionTreeClassifier(), n_estimators=50, random_state=42)
bagging.fit(X_train, y_train)
b_acc = accuracy_score(y_test, bagging.predict(X_test))

# Boosting
boosting = GradientBoostingClassifier(n_estimators=100, learning_rate=0.1,
random_state=42)
boosting.fit(X_train, y_train)
gb_acc = accuracy_score(y_test, boosting.predict(X_test))

print(f'Bagging  Accuracy: {b_acc:.4f}')
print(f'Boosting Accuracy: {gb_acc:.4f}')
print('\nGradient Boosting Report:')
print(classification_report(y_test, boosting.predict(X_test)))
```